STAT 3080: From Data to Knowledge

Jeffrey Woo

2026-08-11

Contents

Preface

Course Overview and Goals

This course focuses on a simulation-based inference approach to concepts taught in elementary statistics. It focuses on using simulations to investigate the operating characteristics of common inferential procedures in statistics, so you can make practical decisions on when to use these procedures to learn about the data at hand.

For example, you may have heard that a hypothesis test for the mean from one sample assumes the data come from a normal distribution, or that the sample size is "large enough". You may have wondered what constitutes being "large enough", and you may have learned or read that a sample size of 25 or 30 will be considered "large enough". The true answer is actually "it depends", as various other factors play a role.

You will learn to design and run simulations to develop a more nuanced understanding of how reliable these inferential procedures are when the assumptions they are based on are not met. It turns out that there are mathematical proofs that can answer such questions, but they are usually fairly complicated, so we will use simulations in place of these mathematical proofs.

In order to design such simulations, a fair amount of knowledge with coding using R will be needed, which you will learn in the first portion of this course. This course is not a programming course; it is simulation-based inference using R.

Upon successful completion of this course, you will be able to:

1. Use simulation to study operating characteristics of statistical methodologies.
2. Understand and apply appropriate statistical methodologies under various data scenarios.
3. Use simulation to estimate statistical quantities which are difficult to compute.
4. Use simulation to assess the quality of statistical estimators.

5. Communicate results from simulations in writing, based on principles of reproducible research.

Background Needed for the Course

The official prerequisites are:

1. Intro statistics (One of STAT 1100, STAT 1120, STAT 2020, STAT 2120, STAT 3120, APMA 3110, APMA 3120).
2. Intro programming (One of STAT 1601, STAT 1602, STAT 3250, CS 1110, CS 1111, CS 1112, CS 1113).

The course is presented assuming you are familiar with inferential procedures associated with one- or two-sample means and proportions, as well as basic programming skills. The course will not cover these.

Required Software

The software introduced and featured in this course is R. RStudio is a software that runs R with additional user-friendly features. RStudio will be used for all in-class R demonstrations. Both R and RStudio are free for download.

- R will need to be downloaded first.
- Then downloaded RStudio.

You are **highly encouraged** to use R Markdown to create PDF documents for homeworks and the project, which uses a type-setting platform called LaTeX. LaTeX can be installed by running the following two commands in the R Console:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

Chapters

The chapters for the book is as follows:

- Chapters ??? focus on core R skills needed: .
- Chapters ??? focus on

Reporting Issues with Course Notes

If you find any issues (typos, formatting, etc), please report them at my GitHub page. Please be as specific as you can, by specifying the section and paragraph where the issue is found.

Serious issues that need my immediate attention should be posted on our Canvas page on Piazza or emailed to me.

Chapter 1

R Basics

1.1 Overview of R

R is an open-source programming language and software environment specifically designed for statistical computing, data analysis, and graphical visualization. It is widely used by data scientists, statisticians, and researchers to clean, manipulate, and model data. R is maintained by volunteers. All users can access and modify source code.

While R has a basic default interface, users typically write code using integrated development environments (IDEs) such as RStudio, Positron, VS Code, which can provide additional user-friendly features. RStudio will be used in this course.

1.1.1 RStudio Interface

Open RStudio. You will see three panes:

- **Console** (left): where commands are executed
- **Environment / History** (top-right:) objects you have created and the commands you have run.
- **Files / Plots / Packages / Help** (bottom-right): files in working directory, plots created, packages installed, help page.

We will focus on the console for the time being.

1.2 Commands

In order to get R to do what you want it to do, you will need to write a **command**. R is ready for a new command when you see a `>` in the **console**. If your command is incomplete, R will show a `+` instead of a `>`. You can either

complete the command after the `+` or use the ESC key to abort. You will also see the output of your command (if any) in the console. For example:

```
1+2
```

```
## [1] 3
```

Notice a number in brackets before the answer? The number indicates index position of the first element displayed on that specific line.

R also ignores white space between components of a command, so typing `1+2` is the same as typing `1 + 2`.

```
1 + 2
```

```
## [1] 3
```

You can also add **comments** to R code by using the `#` sign. Everything to the right of the `#` is ignored by R. Comments are useful to explain what has been coded and why to yourself or to collaborators.

```
1+2 ## what is the sum of 1 and 2?
```

```
## [1] 3
```

I **highly discourage** you to enter your code directly into the R console. Code typed into the console is difficult to be saved or edited. Any meaningful work that you will use R for will likely need to be saved and edited.

1.3 Working Directory

Before talking about a better way to enter R commands, you will need to set up a **working directory**. The working directory is the path to the folder on your system (typically, the computer you are using) to store files generated during an R session and any files that you save. This is also the folder where R will search for files.

1.3.1 Finding the working directory

Typing `getwd()` will display the path of the working directory:

```
getwd()
```

```
## [1] "C:/Users/jeffr/Dropbox/UVA/book_stat3080/stat3080"
```

1.3.2 Changing the working directory

To change the working directory, use `setwd()`, with the path place as an argument:

```
setwd("C:/Users/jeffr/Dropbox/UVA/book_stat3080/stat3080/HWs")
```

I do not recommend using `setwd()`:

- Typing the path is fairly cumbersome.
- Your code will not run if you run the code on another computer (e.g. when sharing with a collaborator).

Therefore, I prefer to use RStudio Projects.

1.3.3 RStudio Projects

An **RStudio Project** is a folder with a `.Rproj` file that in it. There are a couple of ways to create an RStudio Project.

1. If you have never created one before:

- Open RStudio.
- Click on File, then New Project.
- A few options will be presented. The options that you will most likely use are:
  - New Directory: use this to start in a fresh folder.
  - Existing Directory: use this to work in an existing folder.
- If you are a creating a new directory, select New Project. Then you will enter the name of your project and choose a path for the folder.

If you open the folder associated with the created working directory, you will find a file with a `.Rproj` extension created (its name should be the name of the project that you gave in the last step above).

Now, close your RStudio. Navigate to the project folder on your computer and double-click the `.Rproj` file. Your working directory will be where this `.Rproj` file is located. You can confirm using `getwd()`, or by looking at the pane called "Files / Plots / Packages / Help". Click on the files tab and it will display the path of your working directory, and you can also see the other files in this folder.
2. If you have created an RStudio Project before and have a file with a `.RProj` extension, you can just copy and paste this file and place it into another folder to make it your working directory. This is the approach I almost exclusively use.

I highly recommend that you create a folder on your computer that is exclusively for this class. You should also create separate subfolders for each assignment.

Key idea: The working directory is the folder on your computer where all files that R generates will be saved and where R will search for files.

Practice: create an RStudio Project on your own.

1.4 R Scripts

You should type R commands in an R **script** instead of the R console. Commands can be saved and edited in an R script.

To create a new R script, click on File, New File, R Script. You will now have a new pane on the top-left with a tab labeled with a name similar to Untitled. You have created an untitled R script.

An R script looks like a notepad. Type your commands into this script. Each command should be a separate line. To execute commands, highlight the commands, and then click on Run or press Ctrl + Enter.

To save your R script, click on the Save icon and name your script. You have now saved an R Script, and it should appear in your working directory.

To open a saved R script, open the `.Rproj` file in the working directory, and you can click on the script under the tab called Files.

1.5 Basic Math Operations

R can be used as a calculator by using the standard arithmetic operators: + - `+`: addition - `-`: subtraction - `*`: multiplication - `/`: division - `^`: power

R is like any calculator in that these operators follow the standard order of operations. Some examples:

```
15+10
12/6
0.7*5
3^3
15+10*3
(15+10)*3
```

1.6 Assignments and Variables

In order to write useful and effective R code, You need to be able to store items to be used later. Anything that is stored is called an object. An object can be a single value resulting from a calculation or a collection of information such as a table of linear regression coefficients.

A **variable** is a memory location for the object, which can be thought of as a label for the object. Variables are created when the object is **assigned** to it, so there is no need to declare the variable first.

Note: the term variable as defined above comes from computer science. In statistics, the term variable means something different (any characteristic that can be measured).

Variable names can consist of letters, numbers, periods, and underscores. There are a few limitations to how these types of characters can be combined. The following are characteristics of all variable names:

1. Variable names cannot begin with a number.
2. Variable names cannot contain spaces.
3. Variable names can begin with a period as long as they are not followed by a number.
4. Since R is case-sensitive, `x` and `X` are different variable names. Wherever possible, it is best practice to avoid using the names of existing functions (ie. mean, sd, log) and predefined variables (ie. pi) to avoid confusion for anyone reading your code and possibly R itself.

It is also best practice to use variable names that are short and descriptive of the information contained in the object.

Once you have decided on a variable name for the object, you will need to assign the object to the variable by using either the assignment operator `<-` or `=` (the first is the more commonly used in R). I would discourage using `=` as an assignment operator as it is also used when specifying values for arguments in functions (see section 1.7.1.)

To store the sum of 10 and 5 and the rounded value of 5.7 (to whole number) into variables called `total1` and `value1` respectively:

```
total1 <- 10+15
value1 <- round(5.7,0)
```

Notice that these variables and their values are displayed in the environment tab in the pane at the top-right.

R will recall or show the object stored in a variable when the variable name is submitted as a command.

```
total1
```

```
## [1] 25
```

```
value1
```

```
## [1] 6
```

Stored objects can be used in subsequent operations. For example:

```
total1/5
```

```
## [1] 5
```

```
total1/value1
```

```
## [1] 4.166667
```

Any object that you would like stored needs to be given a variable name.

```
ratio1 <- total1/value1
```

Variables can be overwritten by storing a new object using the same variable name.

```
ratio1 <- total1/2
```

1.7 Functions

R has an extensive list of existing functions that can be used. Each function has a specific name that is followed by a set of parentheses. Using a function calls a specific collection of code to be executed. Generally the code that will be executed needs specific arguments or inputs to run correctly and the user provides these values in the parentheses that follow the function name.

Functions can be used for basic calculations, for complex procedures, and many other things in between. You can also write your own functions.

R is case-sensitive, so functions that you want to use must be typed exactly as it was defined.

Some common functions are listed below:

```
log(2.7) ## log with base e, sometimes written as ln
exp(1) ## e to power of 1
sqrt(9) ## squareroot of 9
factorial(3) ## 3!
abs(-5) ## absolute value
sum(2,4,6) ## summing up all these numbers
floor(5.7) ## round down to nearest integer
ceiling(5.7) ## round down to nearest integer
round(5.7) ## round to nearest integer
round(5.727,2) ##round to 2 decimal places
```

1.7.1 Documentation

Typing `?` and then the name of a function will open up the help page (called R documentation) associated with that function. The help page will appear under the Help tab in the bottom-right pane. Type `?log` to read the help page for the `log()` function.

The sections of the help page that I find useful:

- Description: describes what the function does.
- Usage: tells us what arguments need to be provided, and if arguments have defaults.
- Arguments: describes the arguments.
- Examples: some examples.

Practice: after reading the documentation for `log()`, use R to find the value of $\log_3(5)$.

Note: In the top-left corner of the help page, the name of the function is shown, along with the package it is from in braces. So the `log()` function comes a package called `base`.

1.7.2 Working with common probability distributions

Some common probability distributions (e.g. normal, binomial, etc.) are defined in R. Each distribution has a general function name, for example:

- norm: normal distribution
- binom: binomial distribution
- t: t distribution
- F: F distribution
- chisq: χ^2 distribution

Each distribution has 4 functions associated with it in R. The letter that precedes the distribution name tells R what specifically to do with that distribution. These letters are:

- `d`: gives the value of the probability density (or mass) function.
- `p`: gives the cumulative probability of the distribution at the point specified.
- `q`: determines the value within the distribution associated with the cumulative probability specified.
- `r`: generates random numbers from the distribution.

1.7.2.1 Example 1: Normal distribution

Suppose $X \sim N(1, 9)$, i.e. a normal distribution with mean 1 and variance 9.

- To find $P(X \leq 2)$, we use:

```
pnorm(2, mean=1, sd=3) ##note that SD is specified, not variance
```

```
## [1] 0.6305587
```

- To find the value of X associated with the 25th percentile:

```
qnorm(0.25, mean=1, sd=3)
```

```
## [1] -1.023469
```

- To simulate 10 random draws from X, we use:

```
rnorm(10, mean=1, sd=3)
```

```
##  [1] -3.1020708 -0.9782687  3.1092734  1.5631652  5.2254111  0.7496556
##  [7] -1.1559587 -1.1416006 -2.8379719 -0.7732955
```

Note: you are likely to get a vector of 10 numbers that are different than the ones shown.

1.7.2.2 Example 2: Binomial distribution

Suppose $Y \sim Bin(10, 0.6)$, i.e. a binomial distribution with $n = 10$ and $p = 0.6$.

- To find $P(Y = 3)$, we use:

```
dbinom(3, size=10, prob=0.6)
```

```
## [1] 0.04246733
```

To find $P(Y \leq 3)$, we use:

```
pbinom(3, size=10, prob=0.6)
```

```
## [1] 0.05476188
```

Practice: Suppose $S \sim \chi^2_5$, i.e. a chi-squared distribution with 5 degrees of freedom.

- Find $P(S > 0.80)$.
- What is the value of S that corresponds to the 90th percentile?

1.8 Packages

The functionality of R is broken up into various packages. There are a number of packages included in an R download, such as base, graphics, stats, utils. These packages are sometimes collectively called "base R packages". An R **package** is a collection of R functions, corresponding documentation, and datasets.

However, statistical methodologies advance, and additional functions beyond those that exist in base R packages are needed. Independent statisticians create packages to solve more modern problems and have contributed them for everyone to use. These packages are sometimes called "contributed packages".

In order to use what is in a contributed package, that package needs to be installed first, and then loaded. You do not need to install or load base R packages.

In order to install packages, you must have an internet connection. Packages only need to be installed once on a computer unless the version of R is updated. We use the function `install.packages()`. As an example, to install the package called `palmerpenguins`:

```
install.packages("palmerpenguins")
```

You can then load the package in order to able to use the functions from that package, by using the `library()` function:

```
library(palmerpenguins)
```

Key idea: packages only need to be installed once (unless you update R). Packages must be loaded prior to an R session using `library()`.

1.9 Types, Data Structures, Classes

In R, **type** refers to how an object is stored in memory. You can refer to this table for a list of types in R. To find the type of an R object, use the `typeof()` function:

```
typeof(total1)
```

```
## [1] "double"
```

Data structures are used to store and organize values. They are classified based on dimensionality and homogeneity.

- A single value is considered to have zero dimensions and is called a scalar.
- A data structure that is either a single column or a single row is considered to have one dimension.
  - If everything recorded in a one-dimensional data structure is of the same type, then it is called a **vector**.
  - If the items recorded in a one-dimensional data structure have varying types, then it is called a **list**.
- A data structure that contains multiple rows and multiple columns is considered to have two dimensions.
  - If the contents of a two-dimensional data structure are homogeneous, then it is called a **matrix**.
  - If the contents of a two-dimensional data structure are heterogeneous, then it is called a **data frame**.
- An **array** is a data structure that is homogeneous with one or more dimensions.

This information is summarized in Table 1.1 below:

Table 1.1: Classification of R Data Structures

Dimension	Homogeneous (Same Type)	Heterogeneous (Diff Type)
One	Vector	List
Two	Matrix	Data Frame
>2	Array	-

Data structures are created either when data are read in from external files or when data are entered manually. Next, we will see how to create them manually.

1.9.1 Vectors

The concatenate function, `c()`, is commonly used to create a vector. The inputs for this function are the data that you would like stored in the vector in the order in which you would like them stored. For example:

```
x <- c(5,10,15,20,25)
x
```

```
## [1]  5 10 15 20 25
```

```
y <- c(3,6,9,12,15)
y
```

```
## [1]  3  6  9 12 15
```

The concatenate function can also be used to add a value to a vector or to combine preexisting vectors.

```
x2 <- c(x,30)
x2
```

```
## [1]  5 10 15 20 25 30
```

```
z <- c(x,y)
z
```

```
##  [1]  5 10 15 20 25  3  6  9 12 15
```

It is important to remember that the order in which the objects are specified is the order in which they will be saved in the vector.

```
c(y,x)
```

```
##  [1]  3  6  9 12 15  5 10 15 20 25
```

1.9.1.1 Pattern vectors

There are times when you may need to create a vector that contains a sequence of numbers. There are a few ways to create this type of vector.

1. If you are wanting to create a vector that contains a sequence of consecutive integers, then you can use the syntax a:b to create a vector that starts at a and increases or decreases by one until b.

```
1:5
```

```
## [1] 1 2 3 4 5
```

```
5:1
```

```
## [1] 5 4 3 2 1
```

```
-1:-5
```

```
## [1] -1 -2 -3 -4 -5
```

2. If you are wanting a sequence where the values have a constant difference other than one, then you can use the sequence function, `seq()`. This function takes three arguments: from, to, by. These inputs specify the first and last number of the sequence as well as the difference between each value in the sequence.

```
seq(from=1, to=5, by=0.5)
```

```
## [1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

There is another optional input for the sequence function: length. The length option specifies the length that you want your vector to be or how many values you want contained in the vector. The sequence function will use this value to determine a constant difference between each of the values that yields the number of values that you specify.

```
seq(1,5, length=9)
```

```
## [1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

3. Another function that creates a useful type of vector is the replicate function, `rep()`. This function takes two arguments: vector and number of replicates. These specify what you would like R to repeat and how many times. Depending on the format of what you provide in the times option, this function will either repeat the given vector a specified number of times or repeat the elements of the vector a specified number of times.

```
rep(1:5, 2)
```

```
##  [1] 1 2 3 4 5 1 2 3 4 5
```

```
rep(1:5, c(1,2,3,2,1))
```

```
## [1] 1 2 2 3 3 3 4 4 5
```

```
rep(1:5, c(2,2,2,2,2))
```

```
##  [1] 1 1 2 2 3 3 4 4 5 5
```

```
rep(1:5, each=2)
```

```
##  [1] 1 1 2 2 3 3 4 4 5 5
```

1.9.2 Matrices

The `matrix()` function can be used to create a matrix. This function takes four arguments: vector, nrow, ncol, and byrow. These arguments instruct R what values to put in the matrix, how many rows and columns the matrix

should have, and whether values should be assigned across the rows or down the columns. The default for the byrow option is false, meaning that the values will be assigned down the columns if the byrow option is not provided.

```
matrix(c(5,10,15,20,25,3,6,9,12,15), nrow=2, ncol=5, byrow=TRUE)
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    5   10   15   20   25
## [2,]    3    6    9   12   15
```

Another way to create a matrix is to use either the `cbind()` or `rbind()` function. Both of these functions use vectors as inputs. The `cbind()` function will bind these vectors together as columns and the `rbind()` function will bind these vectors together as rows.

```
rbind(x,y)
```

```
##   [,1] [,2] [,3] [,4] [,5]
## x    5   10   15   20   25
## y    3    6    9   12   15
```

```
cbind(x,y)
```

```
##       x  y
## [1,]  5  3
## [2,] 10  6
## [3,] 15  9
## [4,] 20 12
## [5,] 25 15
```

As with the concatenate function, R will bind the vectors together in the order you specify.

1.9.3 Classes

Before going into the remaining data structures, you need to learn what classes are. In R, **class** refers to how an object behaves with generic functions. There are 3 main classes in R: character, numeric, and logical.

- Character: Represents text strings, symbols, or words. Must always be enclosed in quotes.
- Numeric: Represents real numbers.
- Logical: Represents Boolean values for binary conditions.

The vectors and matrices created in sections 1.9.1 and 1.9.2 are numeric.

```
class(y)
```

```
## [1] "numeric"
```

To create a logical vector:

```
log_vect <- c(TRUE, FALSE, T, F)
## Note that T and F are equivalent to TRUE and FALSE resp
```

To create a character vector:

```
char_vect <- c("ByOne", "ByTwo", "ByThree", "ByFour", "ByFive")
```

The `class()` function can be used to find out the class of an object, for example:

```
class(log_vect)
```

```
## [1] "logical"
```

Alternatively, functions such as `is.character()`,is.numeric(),is.logical()‘ can be used to test for the class:

```
is.numeric(log_vect)
```

```
## [1] FALSE
```

If a vector or matrix contains multiple types of data, an error message is unlikely to show up. R will modify the data to all be of the same type according to the following hierarchical structure:

1. character
2. numeric
3. logical

```
c(TRUE, 5)
```

```
## [1] 1 5
```

```
c(TRUE, "ByFive", 5)
```

```
## [1] "TRUE"   "ByFive" "5"
```

```
mult_mat <- cbind(char_vect,x,y)
mult_mat
```

```
##      char_vect x    y
## [1,] "ByOne"   "5"  "3"
## [2,] "ByTwo"   "10" "6"
## [3,] "ByThree" "15" "9"
## [4,] "ByFour"  "20" "12"
## [5,] "ByFive"  "25" "15"
```

1.9.3.1 Example 1: Mixing numeric with character

The most common kind of mixing of classes usually involve a data structure containing a mix of numeric and character, for example:

```
mixed <- c(1, 2, "3")
mixed ## notice all characters, since they are in quotes
```

```
## [1] "1" "2" "3"
```

```
class(mixed)
```

```
## [1] "character"
```

Trying to add the numbers in the variable `mixed` results in an error:

```
sum(mixed)
```

In this example, we can **coerce** all entries to become numeric:

```
mixed <- as.numeric(mixed) ## overwrite existing variable
sum(mixed)
```

```
## [1] 6
```

A situation like this example sometimes happens when working with an external data file and the person who entered the data made a mistake. This is a common reason why a mathematical operation fails as one character changes the entire vector to be character.

Note: Coercing an object to change will not always yield a result that makes sense.

1.9.3.2 Example 2: Math operations with logical

Interestingly, math operations can be performed on logicals. `TRUE` and `FALSE` are considered to be 1 and 0 respectively.

```
sum(log_vect)
```

```
## [1] 2
```

```
mean(log_vect)
```

```
## [1] 0.5
```

This will be extremely useful later on when working with simulations, as we often want to evaluate the proportion of times certain conditions are met.

1.9.4 Data Frames

Data frames are commonly used to store data. There is typically a row for each observation and a column for each variable measured on the observations (think of an EXCEL spreadsheet). A data frame can be created using the `data.frame()` function. The arguments to this function are the vectors that you would like to be in the data frame.

```
mult_data <- data.frame(char_vect,x,y)
mult_data
```

```
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
## 5    ByFive 25 15
```

Notice you can also view the data frame by clicking on its name in the Environment tab in the top-right.

1.9.5 Lists

Lists are the most flexible type of object in R. Lists are similar to vectors in that there is one row containing multiple values. However, in a list, those values can be objects of varying types and sizes. A list is created using the list function, `list()`. The arguments specify what you would like in each element of the list.

```
mult_list <- list(mult_data, z, log_vect, 7)
mult_list
```

```
## [[1]]
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
## 5    ByFive 25 15
##
## [[2]]
##  [1]  5 10 15 20 25  3  6  9 12 15
##
## [[3]]
## [1]  TRUE FALSE  TRUE FALSE
##
## [[4]]
## [1] 7
```

There are testing and coercion functions for all of the types of data structures. As with the classes of data, coercion may not yield a valid result. Some examples are shown below:

```
is.vector(x)
as.vector(mult_list)
as.data.frame(mult_mat)
```

1.10 Other Practical Advice with R

1.10.1 The Workspace

When you are ready to exit R, you may be asked whether you want to save the workspace image into a `.RData` file. The workspace is not the same as the R script. It refers to the objects in the current environment. My recommendation is not to save the workspace.

This keeps your environment fresh the next time you open R. You will then re-run your R script to reproduce your results. Such practice is consistent with principles associated with **reproducible research**. An environment that contains objects from previous workspaces will interfere with the current workspace and must be avoided.

A situation where you may want to save the workspace is if your code takes a long time to run, and you want to avoid having to run your code again. This is unlikely to be a situation in this class.

To remove this prompt, go to Tools, then Global Options. Under the General tab on the left, and the Basic tab on top, uncheck all boxes listed under R Sessions, Workspace, and History. For the option Save workspace to .RData on exit, choose never.

If you have saved the workspace before, find the .RData file in your working directory, and delete it.

1.10.2 The Environment

Always check that your environment is empty (clear the environment) whenever you start working with R. This will ensure that previously declared variables will not influence the results from your code.

You should also clear the environment if you edited your code after executing it. You do not want previous mistakes or version of the code to influence your results.

To clear the environment to make it empty:

1. Execute the code `rm(list = ls())`, or
2. Click on the broom icon on the top-right under the Environment tab.

Chapter 2

R Markdown

2.1 Why Use R Markdown

Imagine you are in this scenario: you have a homework assignment where you are supposed to produce data visualizations and answer some questions based on a data set using R. Your teacher needs to assess if you know what code to write and that you are providing your narrative on the R output. Therefore, you need to produce a document that show the following:

1. R code.
2. Relevant graphs.
3. Your interpretations of the graphs in narrative form.

These have to be presented in a cohesive and organized manner.

Providing an R script will not help, as it only shows your R code and you could provide your narrative as comments. However, your graphs are not shown. Your teacher does not want to produce the graphs by running your R code as it will take a lot of time to run code for all students in the class.

You might think of copying and pasting your R code, your graphs from the output, and type you narrative into a word document. This is doable but it will take you more time than you think, and the document is likely to look sloppy.

Is there a tool that will help you weave your code, R output, and your narrative into a neat document? This is what R Markdown does!

Note: R Markdown is not the only tool that allows you to do this. Quarto is another tool. R Markdown and Quarto can be thought of as variants of Markdown. Markdown is a language for formatting plain text. R Markdown and Quarto expand on Markdown.

2.1.1 Reproducible Research

Another reason for using R Markdown is to adhere to **reproducible research** principles. According to Claerbout and Karrenbach (1992), the definition of reproducible research is "Authors provide all the necessary data and the computer codes to run the analysis again, re-creating the results."

Since your document provides the R commands, readers can simply execute the commands to verify your output. Reproducible research improves transparency and builds trust in your work.

Note: The Turing Way provides an overview of the terms reproducible, replicable, robust, and generalizable for research. We will not go into the details for this class but it is worth your time to think about these.

2.1.2 Prerequisites

While you technically need the `rmarkdown` package, you do not need to explicitly install and load it, if you are using RStudio. RStudio performs these in the background when you knit (render) your document.

2.2 Getting Started

To get started with producing your document with R Markdown, open RStudio, then click on File, New File, R Markdown. Select the Document option, and fill in the entries for title, author, and date. For the time being, use HTML as the default output format.

You will see an untitled tab created. Save it. The file will have a `.Rmd` extension, which is the extension for an R Markdown file. There are some directions provided in this file. Feel free to read it.

In general, there are three essential components of an `.Rmd` file:

1. YAML header.
2. Narrative text.
3. Code chunks.

2.2.1 YAML Header

The **YAML header** controls the settings associated with the document. At this point, your YAML header looks like this:

```
---
title: "Untitled"
author: "Jeffrey Woo"
date: "2026-07-23"
output: html_document
---
```

The YAML header is fenced by three dashes, `---`. These three dashes are also called a **delimiter**. A delimiter is a specific character or symbol used to indicate the boundary between separate, independent regions of the file. You can easily change the title, name, and date by changing the values inside the quotes in the YAML.

2.2.2 Narrative Text

You can type your narration below the YAML.

2.2.2.1 Section Headers

Use hashtags `#` followed by a space to create a section header. The more hashtags the smaller the header.

```
# 1st level header
## 2nd level header
### 3rd level header
```

2.2.2.2 Text Styles

- For italics, fence the word(s) with `*`.
- For bold, fence the word(s) with `**`.
- To display short code in a text, fence the code with “‘.

```
*italicize these words*
**bold these words**
`mean(c(4,6,10))`
```

2.2.2.3 Hyperlinks

For hyperlinks, fence the text with `[ ]` then add the URL in parentheses. For example:

```
Check out the official [R Markdown Documentation](https://rmarkdown.rstudio.com).
```

2.2.2.4 Lists

For an unordered bullet point list, start each line with a dash `-`, asterisk `*`, or plus `+` followed by a space.

```
- First item
- Second item
- Third item
```

For a numbered list, start each line with a number followed by a period and a space.

```
1. First item
2. Second item
3. Third item
```

For a nested list, indent the line with two or four spaces (or one tab) and use the list marker.

```
- Main item 1
    - Sub-item 1a
    - Sub-item 1b
```

2.2.3 Code Chunks

Code chunks allow you to embed R code within your document. There are a few ways to insert a code chunk:

1. Ctrl + Alt + I
2. Click the insert code chunk icon (its a green square with a white C). Click on the down arrow and select R.
3. Type the chunk delimiters ```` ```{r} ```` and ```` ``` ````.

You can type in your R commands inside the code chunks.

2.2.3.1 Chunk options

You can customize how the code runs and displays by using chunk options inside the curly braces. There a lot of options; the ones you may use are:

- `message = FALSE`: which hides messages or warnings that R produces from your document.
- `echo = FALSE`: which hides the code, but allows the results from R to be displayed in your document. This should be used if readers do not want the R code.
- `eval = FALSE`: which shows the code but does not execute the code in your document. This is used for displaying example R code. (I use it a lot in these notes)

2.2.3.2 Inline Code

To display the result from R directly inside the narrative text, use inline R code by fencing the R code with single backticks starting with the letter `r`, for example:

```
The average of 2, 5, and 9 is`r mean(c(2, 5, 9))`.
```

You may want to consider using the `round()` function as R tends to print out a lot of decimal places

2.3 Cross-Referencing with R Markdown

Cross-referencing is done in a document to help connect related information to readers. You will often that you will want to connect a narrative with a graphical output, information presented in a table, a math equation, or a citation.

Cross-referencing is not provided directly within the `rmarkdown` package, but via the `bookdown` extension package. The output format in the YAML needs to be `html_document2`, `pdf_document2`, or `word_document2`. For example

```
---
title: "Untitled"
author: "Jeffrey Woo"
date: "2026-07-23"
output: bookdown::html_document2: default
---
```

R Markdown does cross-referencing and automatically numbers them for you. Below, we will go over a simple example of how to label a figure produced by R, and then refer to it in a narrative.

For this example, we will use the `pressure` data set that is loaded in base R. Per its documentation, the data are the temperature in degrees Celsius and vapor pressure of mercury in millimeters (of mercury).

In the code chunk below, notice a couple of items are added inside the curly braces:

1. A label, `plot-pressure` for this example.
2. A caption via `fig.cap`.

```
```{r plot-pressure, fig.cap="Vapor Pressure of Mercury in mm Hg Against Temp in Celcius"}
plot(pressure)  # a scatterplot
```
```

An example narrative about this plot could be: we can see from Figure 2.1 that vapor pressure increases exponentially with temperature.

To cite the figure, type `\@ref(fig:plot-pressure)` in the narrative, using the label in the code chunk.

This is just a simple example. Referencing tables and equations are similar. Details can be found in Xie et al. (2020) Chapter 4.7.

Note: you will learn more about creating plots in a later Section.

2.4 Knitting Your Document

Once you have your document ready, you can **knit** your document. Knitting transforms your plain text `.Rmd` file into a neat document. Click on the Knit

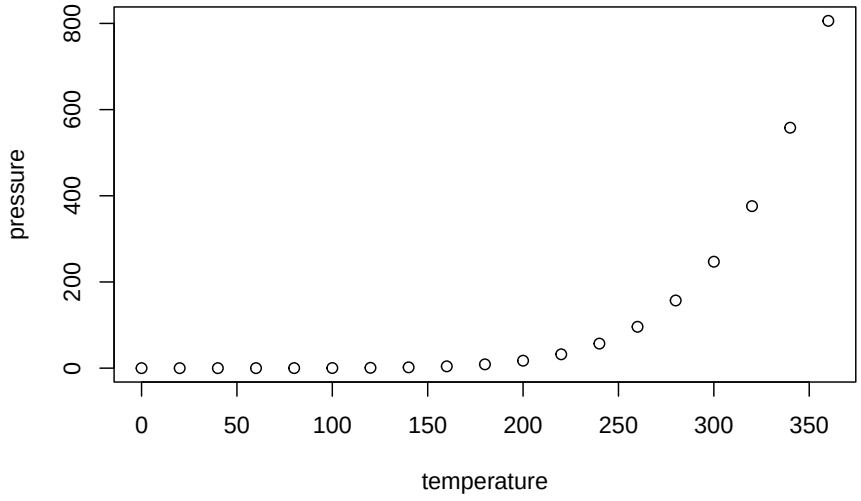

Figure 2.1: Vapor Pressure of Mercury in mm Hg Against Temp in Celcius

button, and your document is produced in the output format chosen at the beginning. This knit button also installs the `rmarkdown` package (if needed) and loads it. This is why you do not have to load it by using the `library()` function.

2.5 Things to Consider

2.5.1 R Script or R Markdown

You might be wondering when should you use an R script and when you should use R Markdown. Personally:

- If I am not going to be sharing the work, or only sharing the work with a small group of collaborators, I am likely to only use an R script.
- If I want to share my work broadly, then I will use an R Markdown. I also start the analysis in an R script, and then transfer the code to an R Markdown, as I find this makes troubleshooting easier.

2.5.2 Troubleshooting

I have seen some students work directly using an `.Rmd` file and bypass an R script. I find troubleshooting error messages difficult in this scenario, as there

are two potential sources of the error:

1. Error in R code inside code chunks.
2. Error in delimiters or options for the YAML, code chunks, inline code.

Personally, I type up and execute my R code in an R script first. Once there are no more errors, I then create my `.Rmd` file and then copy and paste the relevant R code into the code chunks. If an error pops up when knitting, I can rule out errors in the R code and focus on the delimeters and options for the YAML and code chunks.

2.5.3 PDF Output

You will need to produce a PDF file for the project. To do so, you can:

1. Change the default output format to PDF when creating the `.Rmd` file at the beginning.

- To knit the file into a PDF, you need LaTeX, which can be installed by running the following two commands in the R Console:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

2. If this previous step does not work, change the default output format to word when creating the `.Rmd` file at the beginning. Then, convert the word document to a PDF using microsoft word.

2.5.3.1 More Resources for R Markdown

There are a lot more options available with R Markdown. We have only covered the very basics in this unit to get you started with this class. A few other resources with more details can be found below:

- R For Data Science, Ch 27
- R Markdown Cheat Sheet from RStudio
- Cross-Referencing Figures, Equations, and Tables Within Documents.

Chapter 3

Working with Data Structures

3.1 Motivation

Imagine you are working on a data frame in R. For most analyses, common tasks will include subsetting the data frame, performing math operations on the data frame, and computing summaries of some columns of the data frame. You will learn how to perform these tasks in this section.

We will create the same data structures in section 1.9 of the notes to use as examples in this section.

```
x <- c(5,10,15,20,25)
y <- c(3,6,9,12,15)
z <- c(x,y)
log_vect <- c(TRUE, FALSE, T, F)
char_vect <- c("ByOne", "ByTwo", "ByThree", "ByFour", "ByFive")
mult_mat <- cbind(char_vect,x,y)
mult_data <- data.frame(char_vect,x,y)
mult_list <- list(mult_data, z, log_vect, 7)
```

3.2 Subsetting Data Structures

Once a data structure is created and stored, it may be useful in subsequent analyses to subset that data structure. You will learn how to subset data structures using base R functions in this section. Similar operations can be performed using functions from the `dplyr` package, and you will learn these in a later section.

3.2.1 Subsetting with Brackets

3.2.1.1 Subsetting Vectors

The typical notation to **subset** vectors is a set of **brackets**, `[ ]`, used directly after the variable name of the vector. The information given inside the brackets specifies which elements (in terms of index) should be extracted from the vector.

The position index of any vector begins at 1 and uses consecutive positive integers up to the length of the vector. For example, to obtain the 3rd element of the vector `x`:

```
x[3]
```

```
## [1] 15
```

```
x ## for comparison
```

```
## [1]  5 10 15 20 25
```

Note: Indexing in R starts from 1. R is designed for data analysis, statistics, and mathematics, where standard notations begin counting at 1 (e.g. statisticians think in terms of sample size). This is unlike indexing from 0 for other software such as Python and C, who audiences tend to be programmers and software engineers.

Multiple positions can be specified by using a vector in the brackets.

```
x[c(1,3)] ##1st and 3rd elements
```

```
## [1]  5 15
```

```
x[2:3] ##2nd and 3rd elements
```

```
## [1] 10 15
```

If a position is duplicated in the brackets, the corresponding value in the vector will be subset twice.

```
x[c(1,1)] ##1st elements displayed twice
```

```
## [1] 5 5
```

If a negative sign is used in the brackets, the specified value(s) will be omitted instead of included in the subset.

```
x[-1] ##omit element 1
```

```
## [1] 10 15 20 25
```

```
x[-(2:3)] ##omit elements 2, 3
```

```
## [1]  5 20 25
```

```
x[-c(1,4)] ##omit elements 1, 4
```

```
## [1] 10 15 25
```

Instead of specifying the positions that you would like included or excluded from the subset, you can specify a logical vector in the brackets. R will include the elements corresponding to TRUE and exclude the elements corresponding to FALSE.

```
x[c(F,T,T,F,T)]
```

```
## [1] 10 15 25
```

If the number of logical inputs does not match the length of the vector, R will repeat the logical inputs until there is a logical input corresponding to each vector element.

```
x[c(F,T)]
```

```
## [1] 10 20
```

The resulting subset vectors will only be stored if you define a variable name and assign the subset vector to it.

```
evens <- x[c(F,T)]
x
```

```
## [1]  5 10 15 20 25
```

```
evens
```

```
## [1] 10 20
```

The brackets following a vector can also be used to replace or add one or more values in a vector.

```
x2 <- x

x2[6] <- 30 ## add 30 as element 6
x2
```

```
## [1]  5 10 15 20 25 30
```

```
x2[7:9] <- c(35,40,45) ##then add 3 values
x2
```

```
## [1]  5 10 15 20 25 30 35 40 45
```

```
x2[c(1,3)] <- 0 ##change value of elements 1, 3 to 0
x2
```

```
## [1]  0 10  0 20 25 30 35 40 45
```

3.2.1.2 Subsetting Matrices and Data Frames

To subset a matrix or data frame, a similar bracket notation is used. However, because matrices and data frames are two-dimensional structures, two arguments are required in the brackets. The first argument references the row index and the second input references the column index.

Note: subsetting data frames is very common when analyzing data, as data are usually stored in data frames.

The examples below are applied to a data frame. Subsetting matrices work in the same way.

```
mult_data
```

```
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
## 5    ByFive 25 15
```

```
## Subsetting the value in the first row and second column
mult_data[1,2]
```

```
## [1] 5
```

```
## Subsetting the values in the first two rows and first two columns
mult_data[1:2,1:2]
```

```
##   char_vect  x
## 1     ByOne  5
## 2     ByTwo 10
```

```
## Subsetting the values in rows 1, 4 and columns 2, 3
mult_data[c(1,4),2:3]
```

```
##    x  y
## 1  5  3
## 4 20 12
```

```
## Subsetting the entire first row.
## Comma as the 2nd argument means want all columns
mult_data[1,]
```

```
##   char_vect x y
## 1     ByOne 5 3
```

```
## Subsetting the entire second column
## Comma as the 1st argument means want all rows
mult_data[,2]
```

```
## [1]  5 10 15 20 25
```

```
## Subsetting all but the first row
mult_data[-1,]
```

```
##   char_vect  x  y
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
## 5    ByFive 25 15
```

```
## Subsetting all but the third column
mult_data[,-3]
```

```
##   char_vect  x
## 1     ByOne  5
## 2     ByTwo 10
## 3   ByThree 15
## 4    ByFour 20
## 5    ByFive 25
```

```
## Subsetting the third column twice
mult_data[,c(3,3)]
```

```
##    y y.1
## 1  3   3
## 2  6   6
## 3  9   9
## 4 12  12
## 5 15  15
```

```
## Subsetting columns 1, 2, 3 and duplicating the third column
mult_data[,c(1:3,3)]
```

```
##   char_vect  x  y y.1
## 1     ByOne  5  3   3
## 2     ByTwo 10  6   6
## 3   ByThree 15  9   9
## 4    ByFour 20 12  12
## 5    ByFive 25 15  15
```

3.2.1.3 Subsetting Lists

A list is subset to any of its elements by using **double brackets**, `[[ ]]`.

```
mult_list[[3]]
```

```
## [1]  TRUE FALSE  TRUE FALSE
```

Double brackets are limited to one value. In order to subset multiple elements of a list, single brackets can be used.

```
mult_list[2:3]
```

```
## [[1]]
##  [1]  5 10 15 20 25  3  6  9 12 15
##
## [[2]]
## [1]  TRUE FALSE  TRUE FALSE
```

3.2.2 Subsetting with Labels

3.2.2.1 Label Names with Data Structures

Take a look at the output for `mult_data`. Notice there is an extra row on top (as if they are headings or labels for the columns) and an extra column (as if they are labels for the rows).

```
mult_data
```

```
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
## 5    ByFive 25 15
```

Descriptive names can be added to columns, rows, or values within data structures. For data frames and lists, column names can be added while creating the data structure.

```
mult_data2 <- data.frame(Factor=char_vect, Fives=x, Threes=y)
mult_data2
```

```
##    Factor Fives Threes
## 1   ByOne     5      3
## 2   ByTwo    10      6
## 3 ByThree    15      9
## 4  ByFour    20     12
## 5  ByFive    25     15
```

```
mult_list2 <- list(Factor.Table=mult_data, FiveThree=z,
                   Logical.values=log_vect, DaysPerWeek=7)
mult_list2
```

```
## $Factor.Table
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
```

```
## 5    ByFive 25 15
##
## $FiveThree
##  [1]  5 10 15 20 25  3  6  9 12 15
##
## $Logical.values
## [1]  TRUE FALSE  TRUE FALSE
##
## $DaysPerWeek
## [1] 7
```

Column, row, or value **labels** can also be added to an existing data structure using one or more of the following functions: `names()`, `rownames()`, and `colnames()`. The appropriate function(s) to use depends on the type of data structure needing label names. This information is summarized in Table 3.1 below:

Table 3.1: Labels for R Data Structures

Data Structure	`names()`	`colnames()`	`rownames()`
Vector	Elements	-	-
List	List items	-	-
Matrix	-	Column names	Row names
Data frame	Column names	Column names	Row names

My advice: use `names()` for one-dimensional data structures, `colnames()` and `rownames()` for two-dimensional data structures. Examples are displayed below:

```
x ## For comparison
```

```
## [1]  5 10 15 20 25
```

```
## Labeling values in a vector
names(x) <- char_vect
x
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##       5      10      15      20      25
```

```
## Labeling rows and columns in a matrix
mult_mat2 <- cbind(x,y)
rownames(mult_mat2) <- char_vect
colnames(mult_mat2) <- c("Fives","Threes")
mult_mat2
```

```
##         Fives Threes
## ByOne       5      3
## ByTwo      10      6
```

```
## ByThree    15      9
## ByFour     20     12
## ByFive     25     15
```

```
## Labeling columns in a data frame
colnames(mult_data) <- c("Factor","Fives","Threes")
mult_data
```

```
##    Factor Fives Threes
## 1   ByOne     5      3
## 2   ByTwo    10      6
## 3 ByThree    15      9
## 4  ByFour    20     12
## 5  ByFive    25     15
```

```
## Labeling values in a list
names(mult_list) <- c("Factor.Table", "FiveThree", "Logical.values",
                      "DaysPerWeek")
mult_list
```

```
## $Factor.Table
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
## 5    ByFive 25 15
##
## $FiveThree
##  [1]  5 10 15 20 25  3  6  9 12 15
##
## $Logical.values
## [1]  TRUE FALSE  TRUE FALSE
##
## $DaysPerWeek
## [1] 7
```

The `names()` function can also be used to display the names of a data structure.

```
names(x)
```

```
## [1] "ByOne"   "ByTwo"   "ByThree" "ByFour"  "ByFive"
```

```
names(mult_data)
```

```
## [1] "Factor" "Fives"  "Threes"
```

```
names(mult_list)
```

```
## [1] "Factor.Table"   "FiveThree"      "Logical.values" "DaysPerWeek"
```

3.2.2.2 Subsetting with Labels

Data structures with labels can be subsetted using their labels. The name of the column or element can be used within the brackets or after the $ symbol.

```
mult_data[,"Fives"]
```

```
## [1]  5 10 15 20 25
```

```
mult_data$Fives
```

```
## [1]  5 10 15 20 25
```

```
mult_list[["Logical.values"]]
```

```
## [1]  TRUE FALSE  TRUE FALSE
```

```
mult_list$Logical.values
```

```
## [1]  TRUE FALSE  TRUE FALSE
```

3.2.3 Subsetting with Logical Expressions

3.2.3.1 Logical Expressions

It may be useful to use a **logical expression** to determine which, if any, elements in a vector have certain qualities. When a logical expression is applied to a vector, the result will be a vector containing TRUE where the condition is satisfied and a FALSE where the condition is not satisfied.

```
## Are any elements greater than 15?
x > 15
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##   FALSE   FALSE   FALSE    TRUE    TRUE
```

```
## Are any elements greater than or equal to 15?
x >= 15
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##   FALSE   FALSE    TRUE    TRUE    TRUE
```

```
## Are any elements exactly equal to 15?
x == 15
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##   FALSE   FALSE    TRUE   FALSE   FALSE
```

```
## Are any elements not equal to 15?
x != 15
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##    TRUE    TRUE   FALSE    TRUE    TRUE
```

Multiple logical expressions can be combined using either **and** or **or**.

```
## Are any elements in x greater than 15 AND the same element in y is equal to 12?
x > 15 & y == 12
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##   FALSE   FALSE   FALSE    TRUE   FALSE
```

```
## Are any elements greater than 15 OR less than 7?
x > 15 | x < 7
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##    TRUE   FALSE   FALSE    TRUE    TRUE
```

3.2.3.2 Subsetting with Logical Expressions

The logical vectors that result from logical expression statements can be used in the brackets to subset a vector just to the values that satisfy the condition.

```
x[x>15]
```

```
## ByFour ByFive
##     20     25
```

Instead of yielding a logical vector for each entry in a vector, the `which()` function yields the indexes in the vector where the condition is satisfied. The `which()` function can also be used in the brackets to subset a vector to just the values that satisfy the condition.

```
which(x>15)
```

```
## ByFour ByFive
##      4      5
```

```
x[which(x>15)]
```

```
## ByFour ByFive
##     20     25
```

Two-dimensional data frames can be subsetting similarly, using the an argument for the row and another for the column.

3.3 Loading Data Sets in R

We will cover two main ways to load data sets in R:

1. Data sets from R.
2. Data set from a local file.

3.3.1 Data Sets from R

Base R comes pre-loaded with some data sets. You can view them by typing `data()`. You can use the specific data set by typing out its name in the list. To view the `airquality` data set:

```
head(airquality)
```

```
##   Ozone Solar.R Wind Temp Month Day
## 1    41     190  7.4   67     5   1
## 2    36     118  8.0   72     5   2
## 3    12     149 12.6   74     5   3
## 4    18     313 11.5   62     5   4
## 5    NA      NA 14.3   56     5   5
## 6    28      NA 14.9   66     5   6
```

You can use the `head()` function to print out the first 6 rows of a data frame instead of displaying the entire data frame. This gives the reader a good idea of the data without cluttering the document.

Notice that this data frame does not appear in the environment. Personally, I like to assign such data frames to an object so it appears on my environment.

```
Data.air <- airquality
```

Data sets could also come from an R package. For example, the `Davis` data set comes from the `carData` package. So install (if needed) and load the package, and type the name of the data set. Again, I prefer to assign the data set to an object so it appears on the environment.

```
library(carData)
Data.Davis <- carData::Davis
```

Notice that I typed `carData::Davis`. This command tells R to go into the `carData` package and use the data set called `Davis`.

You could have typed `Davis` as well. R would have searched all loaded packages and find the data set called `Davis`. This would work fine if there is no other data set or function with the same name. If another package has a data set or function with the same name, there will be a conflict, and R will use the package that was most recently loaded. This is why I recommend typing the name of the package as well.

3.3.2 Data Sets from Local File

A file containing a data set can be easily loaded into the workspace. Be sure the file is in the working directory (see Section 1.3 on how to set the working directory).

If the data file is comma separated (`.csv`), the function `read.csv()` is used to read in the data.

```
read.csv("Data1.csv", header=TRUE)
```

The argument `header=TRUE` tells R that the first row of the file is a label for the columns (header). If the file does not have a header, specify `header=FALSE`.

Another common type of data file is a `.txt` file. In this case, use `read.table()`.

```
## file has headers
read.table("Data1.txt", header=TRUE)
```

3.3.2.1 Factor Variables

If you read the documentation for the `read.table()` function, you may notice an argument called `stringsAsFactors = FALSE`. This prevents text columns from being automatically converted into **categorical factor variables**, ensuring they are imported as plain character vectors instead.

Factor variables group categorical data into sets containing values with the same string. Each set is given a numerical code that represents the corresponding categorical value. Dummy coding is an example of this. When performing some statistical modeling such as linear regression, categorical data need to be factors.

However, the default is set to `FALSE`, as this saves memory, and it is easier to wrangle with data when they are characters and not factors. Wrangling with data is performed prior to modeling; therefore categorical data are converted to factors after data wrangling and when we are ready for modeling.

To coerce a vector to a factor, use the `as.factor()` or `factor()` functions.

3.4 Math Operations on Data Structures

The standard arithmetic operators can be applied to data structures containing numeric values. To perform the same constant arithmetic operation on each element of a numeric data structure, you will use the object name, the arithmetic operation symbol, and the constant.

```
x <- unname(x) ## remove the name for x
x
```

```
## [1]  5 10 15 20 25
```

```
x*2
```

```
## [1] 10 20 30 40 50
```

```
x/5
```

```
## [1] 1 2 3 4 5
```

```
x^3
```

```
## [1]   125  1000  3375  8000 15625
```

You may have related numeric information stored in two data structures of the same length / dimensions and you may want to perform an arithmetic operation on each pair of related values.

```
x
```

```
## [1]  5 10 15 20 25
```

```
y
```

```
## [1]  3  6  9 12 15
```

```
x+y
```

```
## [1]  8 16 24 32 40
```

```
x*y
```

```
## [1]  15  60 135 240 375
```

```
y/x
```

```
## [1] 0.6 0.6 0.6 0.6 0.6
```

3.5 Applying Functions on Data Structures

Functions can be applied to data structures, for example

```
sum(x)
```

```
## [1] 75
```

```
mean(mult_mat2)
```

```
## [1] 12
```

When working with a data frame, we often want to compute some numerical summaries of some or all of the columns. The `apply()` function applies a pre-existing or user-defined function to the data structure in a manner specified by the user. In this unit, we will just cover the most basic application of the `apply()` function. Variants of this function and user-defined functions will be covered in Section 7.

Suppose you are working on the `airquality` data set, that is stored in the object `Data.air`. The first four columns represents different numeric characteristics associated with air quality and weather conditions. You want to find the average value in each column.

The `apply()` function is used for a two-dimensional data structure. It uses three arguments: data, margin, and function. The data argument specifies the matrix or data frame to which the function should be applied, margin specifies whether

you want the function applied to the rows, columns, or individual values, and function specifies the function to be applied.

```
## 2 since we want to apply the mean to the columns
## use 1 if want to apply mean to the row
## applying to columns is more common as each column
## in a data frame represents a characteristic
apply(Data.air[,1:4], 2, mean)
```

```
##     Ozone   Solar.R      Wind      Temp
##        NA        NA  9.957516 77.882353
```

Notice the NA for two of the columns? This happens when at least one entry in those columns has a missing value. Missing values in R are denoted by `NA`. If you look at the data frame, you can find some NAs in these columns.

If you want R to ignore values and exclude them in calculating the mean, add an argument `na.rm=TRUE` in the `apply()` function.

```
apply(Data.air[,1:4], 2, mean, na.rm=TRUE)
```

```
##      Ozone    Solar.R       Wind       Temp
##  42.129310 185.931507   9.957516  77.882353
```

Suppose you want to count the number of missing values in the column `Ozone`. The `is.na()` function returns a logical identifying whether an entry is missing or not.

```
## the head function returns only the first 6 entries of a vector
## or first 6 rows of a data frame
head(is.na(Data.air$Ozone))
```

```
## [1] FALSE FALSE FALSE FALSE  TRUE FALSE
```

Recall that `TRUE` and `FALSE` are considered to be 1 and 0 respectively. So you can find the number of missing values for `Ozone` with

```
sum(is.na(Data.air$Ozone))
```

```
## [1] 37
```

Chapter 4

Functions

4.1 Motivation

Imagine you are using R to work on a project. You may notice that you have copied and pasted blocks of code in a number of places of your code, with minor tweaks in the value of a particular variable. You will likely be able to replace these blocks of code with a function. A **function** is a reusable block of code designed to perform a specific task. Copying and pasting blocks of code in a number of places is sometimes called **hard-coding**. A benefit of using functions is reducing copying and pasting errors. Manually changing the value of a variable across different blocks of code often leads to mistakes.

You have been using functions that come from base R, or come with other packages. You will now learn how create your own functions.

Functions are defined with three main components:

- the function **name**,
- the **arguments** that the function will take,
- and the **body**, which is the code that should be executed when the function is called.

4.2 Basic Syntax

Suppose you want to convert temperature from Celsius to Fahrenheit in your code. You can create a function in the following manner:

```
CtoF <- function(celsius) {
  celsius * (9/5) + 32
}
```

In this particular example, the function name is `CtoF()`. It only takes in one argument named `celsius`. The value of the argument is the value that can change each time the function is used. We then use an open brace, press enter to enter the body of the function, and then press enter and then use a close brace. The body of the function are the commands that are executed when the function is used.

To use this function to convert 27 degrees Celsius to Fahrenheit

```
CtoF(27)
```

```
## [1] 80.6
```

In this example, you can think of the following sequence of events happening when executing the function named `CtoF(27)`:

- The value of 27 is assigned to `celsius`,
- the math operations as written in the body are performed on `celsius`,
- the value after the operations is returned from the function,
- the value of 27 is no longer stored in `celsius`. In fact, `celsius` is not stored in the environment.

Note: when creating your own function, it is best to avoid using naming your function or arguments with words that are associated with functions in base R.

4.3 Returning a Value

A function in R returns the value of its last evaluated expression. You could explicitly use `return()` in the body to clearly state what you want returned. The function below performs identically to `CtoF()`:

```
## explicit return
CtoFexp <- function(celsius) {
  fahrenheit <- celsius * (9/5) + 32
  return(fahrenheit)
}

CtoFexp(27)
```

```
## [1] 80.6
```

In R, `return()` is not needed when returning a single object, which is unlike other programming languages. If returning multiple objects, `return()` should be used, see Section 4.6.

4.4 Arguments and Default Values

A function can take more than one argument (or even zero arguments such as `getwd()`), and any number of arguments can have a **default** value. Suppose

we want to convert Celsius to Fahrenheit, but also want to function to round to the nearest whole number, by default.

```
CtoFrounded <- function(celsius, decimals = 0) {
  fahrenheit <- celsius * (9/5) + 32
  round(fahrenheit, decimals)
}

CtoFrounded(27)
```

```
## [1] 81
```

```
CtoFrounded(27, decimals = 1)
```

```
## [1] 80.6
```

In the function `CtoFrounded()`, a second argument appears, `decimals`, and is set to 0 by default with the equal sign. This argument does not need to be entered if you want to use the default value. The first argument `celsius` is not set to any default, and must be specified when calling the function.

You can also use `CtoFrounded(27, 1)` to get 27 degrees Celsius converted to 1 decimal place. The numbers entered in the function must be in the same order as the order of the arguments when the function is created.

```
CtoFrounded(27, 1)
```

```
## [1] 80.6
```

Alternatively, you can use `CtoFrounded(decimals = 1, celsius = 27)`. The order does not matter if you specify the names of the arguments.

```
CtoFrounded(decimals = 1, celsius = 27)
```

```
## [1] 80.6
```

You can also decide to have no defaults for any argument at all

```
CtoFnodef <- function(celsius, decimals) {
  fahrenheit <- celsius * (9/5) + 32
  round(fahrenheit, decimals)
}

CtoFnodef(27, 1)
```

```
## [1] 80.6
```

4.5 Argument Validation

Ideally, your function should check if the argument entered satisfies certain conditions, and outputs either an error message if the argument does not work,

or a warning message if the argument is suspicious. Consider out function `CtoF()`. This is called **argument validation**. To convert Celsius to Fahrenheit, the argument entered must be numeric, and the numeric should be within a reasonable range for temperatures.

```
CtoFcheck <- function(celsius) {
  if (!is.numeric(celsius)) {
    stop("celsius must be numeric ")
  }
  if (celsius < -100 | celsius > 100){
    warning("Is celsius within acceptable range?")
  }
  celsius * (9/5) + 32
}

CtoFcheck("blah")
```

```
## Error in CtoFcheck("blah"): celsius must be numeric
```

```
CtoFcheck(250)
```

```
## Warning in CtoFcheck(250): Is celsius within acceptable range?
```

```
## [1] 482
```

The `stop()` function stops execution, while `warning()` will print out the warning message but still executes the code and prints out the result.

Notice `if()` statements are used here. Basically, an `if()` statement checks whether a certain condition is TRUE, if so, the code in the curly braces get executed. If FALSE, the code in the curly braces is skipped, and code outside the curly braces get executed. We will cover `if()` statements in more detail in in Section 5.

When I write functions, I normally add the parts regarding argument validation last. I would test that my function works with valid arguments, then add the argument validation parts, then test with invalid arguments.

4.6 Return Multiple Objects

R functions return a single object. If you need to return multiple objects, you can return those objects into a named list. For example,

```
# Returning multiple items using a list
get_summary_stats <- function(vec) {
  my_mean <- mean(vec)
  my_sd   <- sd(vec)

  # Return objects as a list and name them
```

```
  return(list(mean = my_mean, sd = my_sd))
}

get_summary_stats(c(4,3,1,2,8))
```

```
## $mean
## [1] 3.6
##
## $sd
## [1] 2.701851
```

```
get_summary_stats(c(4,3,1,2,8))$mean
```

```
## [1] 3.6
```

4.7 Scope

Variables inside a function are **local**. This means that the value of a variable is not stored in the environment after the function is run (see Section @ref(#basicFunc) where I described the sequence of events when the function is executed). This means that if there is a variable with the same name outside the function, the variable outside is not overwritten. For example,

```
a <- 3 ## global

toy1 <- function(){
  a <- 10 ## local
  return(a + 20)
}

toy1() ## function uses local a
```

```
## [1] 30
```

```
a ## global untouched
```

```
## [1] 3
```

This means that we do not have to be concerned with names of variables inside a function that we create, as these variables will not overwrite variables with the same name outside the function.

Chapter 5

Control Flow

5.1 Motivation

You may realize that you may need your code to make decisions based on criteria (e.g. argument validation in Section 4.5) or repeat (a set number of times or an unknown amount of times). **Control flow** tools will allow you to do so. The main tools and their functions are:

- Branching with if statements: If a condition is true, execute a block of code. Extensions include the possibility of running another block of code if the condition is false.
- Iterate with for loops: repeat a block of code a fixed, known number of times.
- Iterate with while loops: repeat unknown number of times until a condition is no longer met.

It may be worth reviewing logical expressions in Section 3.2.3.1 as they are used a lot in control flows. Do note that the examples returned logical vectors. Pay attention to the `&` and`|` operators. In these examples, the operators were performed element-wise, and are useful when subsetting or modifying a data structure.

However, in control flows, we want logical expressions to return an output of length 1 (a scalar), so you should use the **scalar operators** `&&` for and and `||` for or.

5.2 Branching

5.2.1 If Statement

Branching with an **if statement** allows our code to execute only if a certain condition is met. If the condition is not met, nothing is executed. For example

```
a <- 1

if (a > 0){
  print("a is positive")
}
```

```
## [1] "a is positive"
```

The parenthesis after the `if` statement is the condition that has to be met for the code to be executed. In this example, the condition is met since `a` is equal to 1, so `a > 0` returns `TRUE`, and the code in the braces is executed. However, if have the following, notice nothing gets printed.

```
a <- -9

if (a > 0){
  print("a is positive")
}
```

Since `a` is equal to -9, the condition `a > 0` is `FALSE`, so the code inside the braces is not executed.

5.2.2 If-Else Statement

If you want to run another block of code when the condition is not met, add an `else` statement that runs when the condition is not met. This is called an **if-else statement**: it branches out based on two conditions.

```
## a is still -9

if (a > 0){
  print("a is positive")
} else {
  print("a is 0 or negative")
}
```

```
## [1] "a is 0 or negative"
```

Since the condition `a > 0` is not met, the code inside the other set of braces (after the `else` statement) gets executed.

So, add an `else` statement if you have two conditions or branches.

5.2.3 Else If

If there are more than two conditions or branches, use `else if` statements between the `if` statement at the beginning and an `else` statement at the end.

```
## a is still -9

if (a > 0){
  print("a is positive")
} else if (a == 0) {
  print("a is 0")
} else {
  print("a is negative")
}
```

```
## [1] "a is negative"
```

The condition `a > 0` is not met, so the code in its braces is not run. The next condition `a == 0` is not met, so the code in its braces is not run. Therefore, the code in the braces after the `else` is executed.

5.2.4 Vectorized Else-If

The `ifelse()` function is a vectorized version of an else-if statement. If applies an else-if statement to each element in a vector, and returns a vector with the same length. For example, I have a number of students in class, and they pass if they score at least 60 on a test, other wise they fail.

```
scores <- c(80, 90, 40, 95, 55)

ifelse(scores >= 60, "pass", "fail")
```

```
## [1] "pass" "pass" "fail" "pass" "fail"
```

```
## condition applied to vector first, then
## what to return if condition is true, then
## what to return if condition is false
```

5.3 Iterating

5.3.1 For Loops

A for loop iterates a block of code for each element in a sequence. As a simple example, suppose I want to print out the values of n^2 for $n = 1, 2, 3, \cdots, 10$:

```
for (n in 1:10){
  print (n^2)
}
```

```
## [1] 1
## [1] 4
## [1] 9
## [1] 16
## [1] 25
## [1] 36
## [1] 49
## [1] 64
## [1] 81
## [1] 100
```

Take a closer look at this for loop. A for loop is made up of 5 components:

1. The for statement.
2. The index variable. In this example, this is `n`. This value changes at each iteration of the loop.
3. The in statement.
4. The vector of values for which the for loop should run, i.e. the values of the variable `n` that should be used. In this example, the values of `n` should be integers 1 to 10.
5. the code to be repeated.

The way to think about how this loops runs, is to:

- Run the code for the first value of the index variable, which is 1, so $1^2 = 1$ is printed.
- When this code if finished for the value of 1, move on to the next value of the index variable, which is 2, so $2^2 = 4$ is printed.
- When this code if finished for the value of 2, move on to the next value of the index variable, which is 3, so $3^2 = 9$ is printed.
- Keep iterating until the last value of the index variable, which is 10, so $10^2 = 100$ is printed.

If values are being calculated in the loop that need to be saved, it is best practice to create a data structure of an appropriate size to record the values during the loop. This practice is called **pre-allocation**. This is preferred to creating a data structure with unknown size and have the data structure grow as the loop is run, as pre-allocation is more efficient.

```
## create a vector to store the 10 squared values
## initialize each entry in the vector to be NA to start
## we have to fill the vector with something at the start
squaredVals <- rep(NA, 10)

for (n in 1:10){
  squaredVals[n] <- (n^2)
  ## store the value of n^2 in each entry of the vector
  ## replacing the original value of 0 in each entry
```

```
}
```

Ideas associated with a for loop, i.e. iterating a block of code a set number of times, is used a lot in simulations. As mentioned in the preface, simulations are used to estimate statistical quantities that may be difficult to compute. It turns out that there are more efficient ways to run simulations.

5.3.2 While Loops

A while loop iterates a block of code until some condition is met. Typically, number of iterations is unknown. While loops are much less common in practice though. We will just use a simple example to illustrate this idea.

Suppose you play a gambling game where you start with $5. Each time you play the game, the amount of money you lose is a random number that comes from a normal distribution with mean 2, and standard deviation 1. You keep playing the game until you run out of cash. We want to simulate the number of times you play the game, and record the amount of cash after each time you play the game.

```
## set.seed() for reproducibility due to random nature
## of random loss each time you play
set.seed(10)

cash <- 5 ## initial amount of cash

##store amount of cash at each play.
##start with empty vector as we don't know
##how many times you will play
##don't do this with for loop as pre-allocation
##is more efficient than starting with empty vector
storeCash <- c()
numPlays <- 0 ## counter to count number of times you play

while (cash > 0) {
  cash <- cash - rnorm(1, mean = 2, sd =1)
  numPlays <- numPlays + 1
  storeCash[numPlays] <- cash
}

numPlays
```

```
## [1] 4
```

```
storeCash
```

```
## [1]  2.9812538  1.1655064  0.5368369 -0.8639954
```

5.4 Vectorization in R

Vectorization in R is a feature of R where operations are applied to an entire vector or data structure at once, rather than iterating through each individual element one by one using a traditional loop. You have seen this feature in the examples in Section 3.4. Vectorization makes performing math operations and functions on data structures efficient.

Consider the example in Section 5.3.1. We have a vector `1:10` and we want to square each value. Instead of squaring each entry in the vector one by one for a loop, we can easily type `(1:10)^2` to get the same result.

As your code gets longer and more complex, making it more efficient with vectorization and pre-allocation will make a difference in how long your code runs. Another way of making code more efficient in R is using the `apply` family of functions: you have seen a simple example in Section 3.5, and you will learn more about this family of functions in Section 7.

Chapter 6

Simulations

6.1 Motivation

When we are analyzing data, there is always going to be some degree of uncertainty, as there is **randomness** in a lot of phenomena that we observe in our world. An example that is common in statistics comes from the field of sample surveys. A poll is conducted to see what proportion of voters will vote each candidate in an upcoming election. Even with a well-designed poll and a large sample size, the value of the sample proportion that will vote for a candidate has some randomness to it. If we get another random sample, the sample proportion is likely to be different (but hopefully close). The act of obtaining a random sample, or **sampling**, from a population introduces some randomness.

Statisticians and data scientists often want to investigate the degree of randomness involved in what is being studied. Sometimes, this can be done mathematically, but oftentimes, the mathematics is extremely challenging. You may also realize in the previous paragraph that one way of studying the randomness in polls is to repeatedly obtain random samples. But it is practically unfeasible to repeatedly go out in the field to collect data.

This is why simulations are used. We do not need to solve complex mathematics, and we can simulate the studies many times (in the thousands, at least) with computers, saving time and money than if we physically ran these studies many times.

You will see how sampling introduces randomness and then learn how to create simulations to quantify the degree of randomness or uncertainty.

6.2 Sampling

The `sample()` function in R randomly draws a selected number of items from a list of items (the list is usually a vector), and the probabilities of each item being selected are the same for all items. A simple example of randomness is tossing a fair coin a set number of times (equal chance of landing heads or tails).

```
fairCoin <- c("heads", "tails")
sample(fairCoin, size = 5, replace = TRUE)
```

```
## [1] "heads" "heads" "heads" "tails" "heads"
```

The `sample()` function has three main arguments:

- A vector of values from which to sample.
- How many values from the vector to draw.
- If you want to sample with or without replacement.

Sampling with replacement means that when we sample a value from the vector, the value gets returned to the vector and can be sampled again.

Sampling without replacement means that means that when we sample a value from the vector, the value does not get returned to the vector and cannot be sampled again.

Run the code in the block above a few times. You will notice that the returned vector is not always the same, so the result is unpredictable. This is unlike most of the earlier code you have seen in the course notes.

Another use of sampling is in model validation (i.e. assessing model accuracy). The original data set is randomly split into two mutually exclusive data sets: one called training and the other called test. The training set is used to build the model, and model assessment is done on the test set. Suppose we want to split the `Davis` data set from the `carData` package.

```
library(carData)
Data.Davis <- carData::Davis
n <- dim(Data.Davis)[1]
## when dim() is applied to a dataframe, two values get returned
## the num of rows and number of columns. The number of rows is
## equal to the sample size

trainInd <- sample(1:n, size = ceiling(n/2), replace = FALSE)
## each observation belongs to a row. So randomly select half of
## row indexes without replacement so the two sets are mutually ## exclusive

## split data into training and test
trainData <- Data.Davis[trainInd,]
testData <- Data.Davis[-trainInd,]
```

You have also seen sampling before in Section @red(distributions), where functions such as `rnorm()` and `rbinom()` draw random numbers from known probability distributions such as the normal and binomial distributions. Try typing `rnorm(10)` a few times. You should get a vector containing 10 random draws from a standard normal, and the vectors are not always the same.

6.3 Replicate

You have seen how sampling is inherently random. In order to quantify the uncertainty introduced through sampling, we want to be able to repeat, or **replicate**, the sampling a lot of times.

The `replicate()` function evaluates a given function the specified number of times and returns a matrix with a column containing the results from each replication. There are two arguments: the number of replications and the function to be replicated.

Going back to the coin tossing example, suppose we want to toss a fair coin 5 times, and replicate this experiment for 6 replicates.

```
replicate(6, sample(fairCoin, size = 5, replace = TRUE))
```

```
##      [,1]    [,2]    [,3]    [,4]    [,5]    [,6]
## [1,] "heads" "tails" "heads" "heads" "tails" "tails"
## [2,] "heads" "tails" "heads" "heads" "tails" "tails"
## [3,] "tails" "heads" "tails" "heads" "heads" "heads"
## [4,] "tails" "heads" "tails" "tails" "tails" "tails"
## [5,] "heads" "tails" "tails" "tails" "heads" "heads"
```

From section 5, you may think of using a for loop to execute these steps. A for loop could do the same, but the code is longer.

6.4 Simulation

According to Bonnell and Ogihara (2024), a simulation consists of three main steps:

1. Decide what to simulate. What process do we want to imitate?
2. Figure out how to simulate one value.
3. Replicate step 2 many times.

In the previous block of code, we see these steps in action:

- We want to imitate the results from tossing a fair coin 5 times.
- We use the `sample()` function to simulate one "value", which is a vector storing the outcomes from 5 tosses of the coin.
- We use the `replicate()` to repeat the previous step for a total of 6 replicates.

6.4.1 Worked Example: Sampling Distribution of the Sample Mean

In introductory statistics, you learned that if $X \sim N(\mu, \sigma^2)$, then $\bar{X} \sim N(\mu, \frac{\sigma^2}{n})$. This result is sometimes called the sampling distribution of the sample mean, and can be proved mathematically. However, such proofs are not the scope of this class.

This means that if we have data from a $N(\mu, \sigma^2)$ distribution, then the distribution of the sample means from a large number of random samples each of size n is $N(\mu, \frac{\sigma^2}{n})$.

We will see how we can use simulation to illustrate the sampling distribution of the sample mean.

Suppose the data follow a standard normal distribution, i.e. $X \sim N(0, 1)$. The sampling distribution of the sample mean tells us that based on samples each of size 10, $\bar{X}_{10} \sim N(0, \frac{1}{10})$. So the three main steps of this simulation would be:

1. Draw a random sample of 10 from a standard normal distribution, compute the sample mean, and store this value.
2. Use `rnorm(10)` to simulate 10 draws from a standard normal distribution, then compute the sample mean of these 10 draws, and store this value.
3. Repeat step 2 for a large number of times, say for a total of 10,000 replicates.

Step 3 involves using the `replicate()` function, where we supply the number of replicates and a function that gives the result from step 2, i.e. the sample mean from 10 random draws from a standard normal. While there is no function in base R that does this, we can easily write our own function to do this (see Section 4 on how to write functions)!

```
sampMean <- function(number){
  mean(rnorm(number))
}
```

Our created `sampMean()` computes the sample mean of a certain number of draws (that is supplied) from a standard normal distribution. `sampMean()` executes step 2.

For step 3, we can just use `replicate()` on `sampMean()`. Suppose we want 10,000 replicates. We store the value of the sample mean from 10,000 random samples, each with size 10.

```
Reps <- 10000
storeSampMeans <- replicate(Reps, sampMean(10))
```

Intuitively, we know the sample means from each sample are different. We can print out a few of these values to confirm this idea.

```
head(storeSampMeans)
```

```
## [1] -0.28824148  0.56248186  0.02602954  0.30351791  0.29413886  0.92128312
```

If the sampling distribution of the sample mean is correct, we know that $\bar{X}_{10} \sim N(0, \frac{1}{10})$, theoretically. Our simulation should be consistent with this theory.

We can create a density plot of the 10,000 sample means to visualize the uncertainty from sampling. We can see in figure 6.1 that most of the sample means are close to 0, and values further from 0 are less likely to occur. This also looks like a normal distribution.

```
plot(density(storeSampMeans), main="")
```

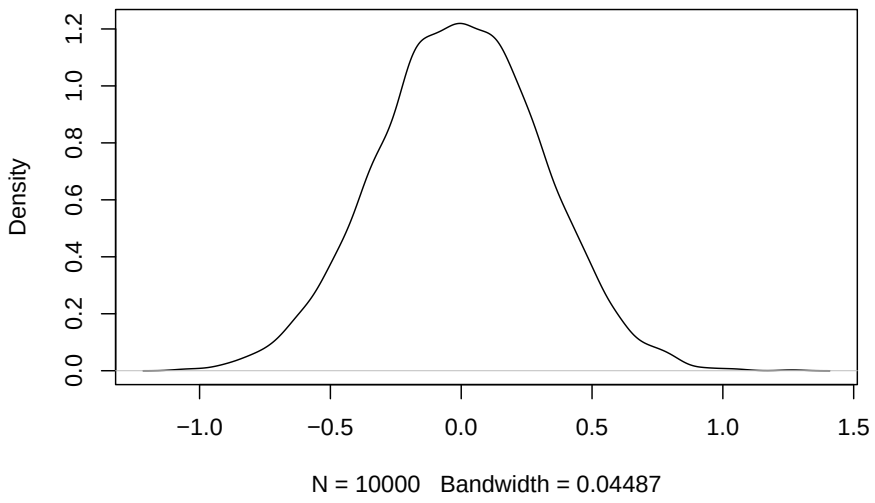

Figure 6.1: Density Plot of Sample Means from 10,000 Random Samples of Size 10

We know that the mean and variance of the 10,000 sample means should be close to 0 and 0.1 theoretically.

```
mean(storeSampMeans)
```

```
## [1] -0.005047077
```

```
var(storeSampMeans)
```

```
## [1] 0.09895951
```

The variance measures how spread out the sample means are, and is considered a measure of the uncertainty associated with the sample mean from samples of size 10.

We have just created a simulation that imitates the sampling distribution of the sample mean when the data are standard normal and each sample has size 10!

Hopefully, this example convinces you that with a large number of replicates, results from a simulation match very closely with theory.

Note: you could have gotten similar results by using a for loop, using the code below. However, the for loop is a bit longer as we need to pre-allocate.

```
## Repeat for total 10,000 times
Reps <- 10000
## pre allocate vector to store the 10,000 sample means
sampleMeansLoop <- c(NA, Reps)

for (i in 1:Reps){
  ## draw 10 numbers from standard normal, find the mean, and store
  sampleMeansLoop[i] <- mean(rnorm(10))
}

plot(density(sampleMeansLoop), main="")
mean(sampleMeansLoop)
var(sampleMeansLoop)
```

In many other situations, mathematical theory can be difficult to prove. For example, what is the sampling distribution of the median from a non-normal population? While finding this result will be challenging mathematically, we can use simulation to get very close to the result.

6.4.2 Considerations with Simulations

In the example above in Section 6.4.1, we compared results from a simulation with the corresponding theoretical values, so we could see that simulations work. However, if we do not know the theoretical values, two questions will come to mind:

- How many replicates do we need? We only know the values from the simulation converge (get close) to the theoretical values as we increase the number of replicates. Using more replicates will make the simulation run longer on your computer.
- Related to the previous question, how close is close enough? How do you know your value from the simulation is close enough to the theoretical value? There is no way of knowing if the theoretical value is unknown.

Chapter 7

Apply Family

7.1 Overview

We often want to apply the same function separately to each element of a vector. R has the `apply` family of functions that implement loops in specialized use cases. This family of functions are useful since it:

- is compact,
- can iterate over different data structures and return collated results, and
- use anonymous functions to improve performance.

This family of functions include:

- `apply()`: Apply a function over the margins of an array.
- `tapply()`: Apply a function over subsets of a vector.
- `lappy()`: Loop over a list and evaluate a function on each item.
- `sapply()`: Similar to `lapply()` but attempt to simplify the result.

7.2 `apply()`

You learned about the `apply()` function in Section 3.5. It is to apply a function over the margins of an array. To reiterate: it uses three arguments: data, margin, and function. The data argument specifies the matrix or data frame to which the function should be applied, margin specifies whether you want the function applied to the rows, columns, or individual values, and function specifies the function to be applied. For example, to find the average weight and height all respondents in the `Davis` data set from the `carData` package:

```
library(carData)
Data.Davis <- carData::Davis
apply(Data.Davis[,2:3], 2, mean)
```

```
## weight height
##  65.80 170.02
```

```
##2 for columns, 1 for rows
```

You may realize that one could use a loop to obtain these values. The `apply()` function is more succinct, although it does not really run faster than a loop (Peng, 2015).

We so often compute sums or means of columns or rows that there functions that produce the same output as equivalent `apply()` versions:

- `rowSums(X)` equivalent to `apply(X, 1, sum)`.
- `rowMeans(X)` equivalent to `apply(X, 1, mean)`.
- `colSums(X)` equivalent to `apply(X, 2, sum)`.
- `colMeans(X)` equivalent to `apply(X, 2, mean)`.

These functions are faster than `apply()` especially when `X` is large (Peng, 2015).

7.3 tapply()

The `tapply()` function is used for a vector of values that can be categorized by a second vector of values. It uses three arguments: data, categorizations, and function. The data input specifies the vector of values that should be grouped and to which the function should be applied, categorizations specifies the category of each corresponding value in the data vector, and function specifies the function to be applied to each resulting group. For example, to find the average weight of female and male respondents in the `Davis` data set.

```
tapply(Davis$weight, Davis$sex, mean)
```

```
##        F        M
## 57.86607 75.89773
```

7.4 lapply() and sapply()

The `lapply()` and `sapply()` functions are both used for lists. Once the specified function is applied, `lapply()` returns a list and `sapply()` attempts to simply the result. They use two arguments: data and function. The data argument specifies the list of elements to which the function should be applied and function specifies the function. If the function has other arguments, they can be specified as additional arguments in `lapply()` and `sapply()`. For example:

```
v1 <- list(a = 1:5, b = rnorm(10))
lapply(v1, mean)
```

```
## $a
## [1] 3
```

```
##
## $b
## [1] -0.1321658
```

```
##get the mean of 1:5 and the mean for 10 draws from standard normal
```

You can use `lapply()` to evaluate a function multiple times each with a different argument. This can be useful if you want to vary the value of an argument in a simulation. For example, to replicate drawing n values from a standard uniform distribution, with $n = 1, 2, 3, 4$.

```
v2 <- 1:4
lapply(v2, runif)
```

```
## [[1]]
## [1] 0.8183945
##
## [[2]]
## [1] 0.10739277 0.09259545
##
## [[3]]
## [1] 0.4691778 0.6536973 0.8917855
##
## [[4]]
## [1] 0.65201199 0.02842239 0.61628714 0.74522073
```

When you pass a function to `lapply()`, `lapply()` takes elements of the list and passes them as the first argument of the function you are applying. In the above example, the first argument of `runif()` is `n`, the number of random draws, and so the elements of the sequence `1:4` are all passed to the `n` argument of `runif()`.

Functions that you pass to `lapply()` may have other arguments. For example, the `runif()` function has a `min` and `max` argument. When these are unspecified, the default values are 0 and 1 respectively, which gives the standard uniform distribution. If these arguments need to be specified, they are passed to `lapply()` next. For example, to replicate drawing n values from a uniform distribution with `min=3` and `max=10`, with $n = 1, 2, 3, 4$.

```
lapply(v2, runif, min = 3, max = 10)
```

```
## [[1]]
## [1] 9.151133
##
## [[2]]
## [1] 8.42458 9.54934
##
## [[3]]
## [1] 4.206172 4.269696 4.661913
##
```

```
## [[4]]
## [1] 9.751580 3.800813 6.711054 9.158739
```

You can also pass your own function to `lapply()`.

Anonymous functions can also be passed to `lapply()` (and other functions in the apply family). Anonymous functions are functions you create that have no variable name and are not stored in the environment. For example, you have a couple of matrices and you want to extract the first row of each one.

```
twoMats <- list(a = matrix(1:4, 2, 2), b = matrix(1:6, 3, 2))

twoMats
```

```
## $a
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
##
## $b
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
```

```
lapply(twoMats, function(extract) { extract[1,] })
```

```
## $a
## [1] 1 3
##
## $b
## [1] 1 4
```

I recommend using anonymous functions only if the function is simple and if you are going to only use it once. Otherwise, create a function and name it before `lapply()`, and pass the function to `lapply()`. For example

```
row1 <- function(extract){
  extract[1,]
}

lapply(twoMats, row1)
```

```
## $a
## [1] 1 3
##
## $b
## [1] 1 4
```

Notice that the function `row1` now appears in the environment.

7.5 sapply()

`sapply()` is similar to `lapply()`, but `sapply()` attempts to simplify the result of `lapply()`.

- If the result is a list where every element is length 1, then a vector is returned.
- If the result is a list where every element is a vector of the same length and greater than 1, a matrix is returned, where each column represents each vector.
- Otherwise, a list is returned, just like `lapply()`.

Look at the output from the following, and compare with the output from `lapply()`.

```
sapply(v1, mean)
```

```
##          a          b
##  3.0000000 -0.1321658
```

```
sapply(twoMats, row1)
```

```
##      a b
## [1,] 1 1
## [2,] 3 4
```

I find working with vectors and matrices easier than working with lists, so I prefer using `sapply()` to `lapply()`.

Chapter 8

Blank Canvas

See Sections **??**, 1.7.1.

See equation (**??**).

See Figure 8.1, and 8.2 and Table **??**.

```
plot(cars)
```

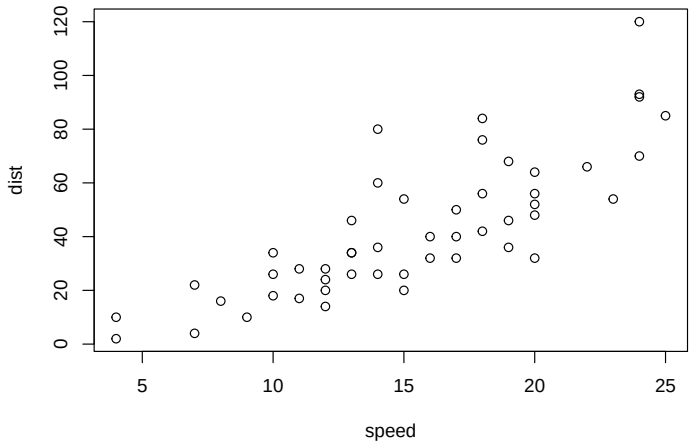

Figure 8.1: Car

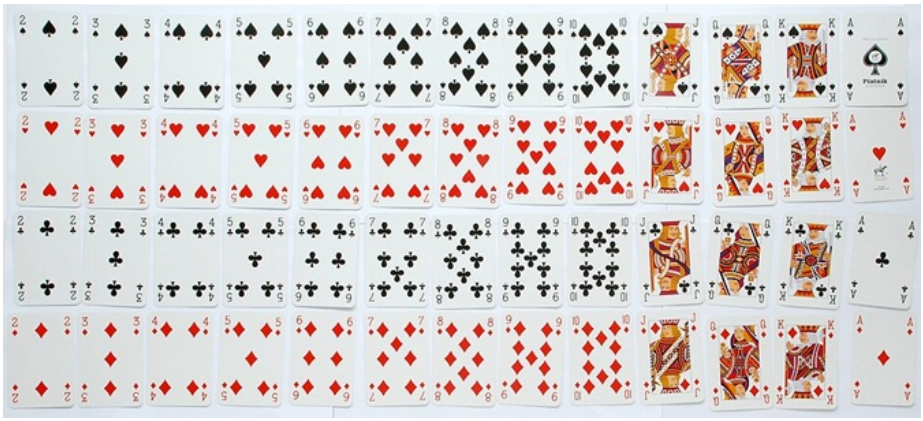

Figure 8.2: Sample Space of Drawing One Card from Standard Deck. Picture from wikipedia

Bibliography

Bonnell, J. and Ogihara, M. (2024). *Exploring Data Science with R and the Tidyverse*. Chapman and Hall/CRC, 1st edition. ISBN 978-1032341705.

Claerbout, F. and Karrenbach, M. (1992). Electronic documents give reproducible research a new meaning. In *SEG Technical Program Expanded Abstracts 1992*.

Peng, R. (2015). *R Programming for Data Science*. Lulu.com, 5th edition. ISBN 978-1365056826.

Xie, Y., Dervieux, C., and Riederer, E. (2020). *R Markdown Cookbook*. Chapman and Hall/CRC, Boca Raton, Florida. ISBN 978-0367563837.